```
module ExpenseTracker
  class API < Sinatra::Base
  end
end
```

This class defines the barest skeleton of a Sinatra app. Now, our tests need to load it. Back in your spec, add the following line to the require section at the top:

04-acceptance-specs/02/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb
```
require_relative '../../app/api'
```

Now, your specs are passing. You may be wondering, "Did we actually test anything? Shouldn't we expect or assert something?"

We'll get to that. Right now, we're verifying only that the POST request completes without crashing the app. Next, we'll actually check that we got a valid response back from the app.

Checking the Response

Rack::Test provides the last_response method for checking HTTP responses. Add the following line inside your spec, right after the post request:

04-acceptance-specs/03/expense_tracker/spec/acceptance/expense_tracker_api_spec.rb
```
expect(last_response.status).to eq(200)
```

You've encountered code like this before, in *Your First Spec*, on page 5. It's an *expectation* fulfilling the role an assertion method would play in other test frameworks.

Together, expect() and to() check a result in order to signal success or failure. They compare a value—in this case, the HTTP status code returned by last_response.status—using a *matcher*. Here, we create a matcher using the eq method, which indicates whether or not the value wrapped by expect equals the provided argument of 200. When we pass the matcher from eq(200) as an argument to the to() method, we'll get a passing or failing result.

This may seem like a lot of moving parts compared to a traditional assertion-style method like assert_equal. Matchers are, however, more powerful and more composable than traditional assertions. We'll see more about them in *Exploring RSpec Expectations*.

Let's look at what happens when we run this code:

```
$ bundle exec rspec
F

Failures:

  1) Expense Tracker API records submitted expenses
```